





Mark state as... ▼

Event Log

Left -&gt; Right

■ success  
BashOperator

React Flow

Activities

dbaver-ce

Apr 29 03:41

DBeaver 26.0.3 - marts

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutoDatawarehouse.duckdbDatawarehouse

Database NaviProjects

Filter connections by name

Datawarehouse.duckdb

Datawarehouse

main

marts

Tables

dim\_customers

dim\_geolocation

dim\_orders

dim\_payments

dim\_products

dim\_reviews

dim\_sellers

fact\_order\_items

fct\_sales\_comprehensive

Views

Indexes

Procedures

Sequences

Data Types

raw

staging

DBEaver Sample Database (SQLite)

Files - General

NameDataSc

Bookmarks

Dashboards

Diagrams

Scripts

PropertiesDiagram

dim\_payments

AZ order\_id

123 payment\_sequential

AZ payment\_type

123 payment\_installments

123 payment\_value

123 total\_payment\_value

dim\_products

AZ product\_id

AZ product\_category\_name

123 product\_name\_lenght

123 product\_description\_lenght

123 product\_photos\_qty

123 product\_weight\_g

123 product\_length\_cm

123 product\_height\_cm

123 product\_width\_cm

dim\_reviews

AZ review\_id

AZ order\_id

123 review\_score

AZ review\_comment\_title

AZ review\_comment\_message

review\_creation\_date

review\_answer\_timestamp

dim\_orders

AZ order\_id

AZ customer\_id

AZ order\_status

purchase\_at

approved\_at

delivered\_carrier\_at

delivered\_customer\_at

estimated\_delivery\_at

fact\_order\_items

AZ order\_item\_key

AZ order\_id

AZ order\_item\_id

AZ product\_id

AZ seller\_id

shipping\_limit\_date

123 price

123 freight\_value

dim\_sellers

AZ seller\_id

123 seller\_zip\_code\_prefix

AZ seller\_city

AZ seller\_state

dim\_customers

AZ customer\_id

AZ customer\_unique\_id

123 customer\_zip\_code\_prefix

AZ customer\_city

AZ customer\_state

dim\_geolocation

123 geolocation\_zip\_code\_prefix

123 geolocation\_lat

123 geolocation\_lng

AZ geolocation\_city

AZ geolocation\_state

RefreshSave ...RevertKeep layout100%

Entity Diagram

EESTen














```
1 from datetime import datetime
2 from pathlib import Path
3 from airflow import DAG
4 from airflow.operators.bash import BashOperator
5 from cosmos import DbtTaskGroup, ProjectConfig, ProfileConfig, ExecutionConfig
6
7 # 1. PATH CONFIGURATION
8 PROJECT_ROOT = Path("/home/khalidabdelrazk/data engineer/e-commerce")
9 DBT_ROOT = PROJECT_ROOT / "dbt" / "dbt_transformations"
10 VENV_PYTHON = PROJECT_ROOT / ".venv" / "bin" / "python"
11 VENV_DBT = PROJECT_ROOT / ".venv" / "bin" / "dbt"
12
13 # 2. PROFILE CONFIGURATION
14 profile_config = ProfileConfig(
15     profile_name="ecommerce_platform",
16     target_name="dev",
17     profiles_yml_filepath=Path("/home/khalidabdelrazk/.dbt/profiles.yml"),
18 )
19
20 with DAG(
21     dag_id="olist_end_to_end_pipeline",
22     start_date=datetime(2024, 1, 1),
23     schedule_interval="@daily",
24     catchup=False,
25 ) as dag:
26
27     # TASK 1: INGESTION
28     ingest_data = BashOperator(
29         task_id="ingest_csv_to_duckdb",
30         cwd=str(PROJECT_ROOT / "ingestion"),
31         bash_command=f"'{VENV_PYTHON}' main.py",
32     )
33
34     # TASK 2: DBT TRANSFORMATIONS
35     dbt_transform = BashOperator(
36         task_id="dbt_run_transformations",
37         cwd=str(DBT_ROOT),
38         bash_command=f"'{VENV_DBT}' run --profiles-dir /home/khalidabdelrazk/.dbt",
39     )
40
41     ingest_data >> dbt_transform
```



```

1  import pandas as pd
2  import functions as fn
3  import config
4  import os
5
6  if __name__ == "__main__":
7      # 1. Initialize DB
8      conn = fn.init_duckdb()
9      fn.create_schema(conn)
10
11     # merged tables
12     merged_df = fn.merge_ingested_data(config)
13
14
15     try:
16         # Pass the table name to make it dynamic
17         latest_ts = fn.get_latest_timestamp(conn, "orders", "order_purchase_timestamp")
18         print(f"Latest timestamp in orders: {latest_ts}")
19     except Exception as e:
20         print(f"Error occurred while fetching latest timestamp for orders: {e}")
21         latest_ts = None
22
23
24     # for col in merged_df.columns:
25     #     if 'timestamp' in col or 'date' in col:
26     #         merged_df[col] = pd.to_datetime(merged_df[col])
27
28     # 2. Process each table
29     for table_name, details in config.TABLES.items():
30         print(f"--- Processing: {table_name} ---")
31
32         # Pull the columns we want to KEEP in the DB
33         final_columns = details["columns"]
34         # The column we use for filtering (order_purchase_timestamp)
35         ts_col = details["timestamp_column"]
36
37         # Get data for this table + the timestamp column for filtering
38         cols_to_pull = final_columns
39         df = merged_df[cols_to_pull].copy()
40
41         # Deduplicate
42         # df.drop_duplicates(subset=details["primary_key"], inplace=True)
43
44
45
46         # 4. Filter for new records BEFORE dropping the TS column
47         if latest_ts:
48             latest_ts = pd.to_datetime(latest_ts)
49             latest_ts = str(latest_ts)
50
51             new_data = df[df[ts_col] > latest_ts]
52             print(f"Found {len(new_data)} new records for {table_name}.")
53         else:
54             new_data = df
55             print(f"Found {len(new_data)} new records for {table_name}.")
56
57         if table_name != "orders":
58             final_columns = [col for col in final_columns if col != ts_col]
59
60         if not new_data.empty:
61             # 5. Define the Final Payload (Only the columns specified in config)
62             # This ensures 'order_purchase_timestamp' only goes into 'orders'
63             # if it was explicitly listed in that table's columns in config.py
64             payload = new_data[final_columns]
65
66             # Create table if first run
67             fn.create_tables(conn, table_name, payload, details["primary_key"])
68
69             # Insert data
70             fn.insert_data(conn, table_name, payload)
71             print(f"Inserted {len(payload)} records into {table_name}.")
72         else:
73             print(f"No new records for {table_name}.")
74
75     conn.close()

```





```
1  TABLES = {
2      "orders": {
3          "file_path": "../data/olist_orders_dataset.csv",
4          "primary_key": "order_id",
5          "timestamp_column": "order_purchase_timestamp",
6          "columns": ["order_id", "customer_id", "order_status", "order_purchase_timestamp", "order_approved_at", "order_delivered_carrier_date", "order_delivered_customer_date", "order_estimated_delivery_date"]
7      },
8      "order_items": {
9          "file_path": "../data/olist_order_items_dataset.csv",
10         "primary_key": "order_item_id",
11         "timestamp_column": "order_purchase_timestamp",
12         "columns": ["order_id", "order_item_id", "product_id", "seller_id", "shipping_limit_date", "price", "freight_value", "order_purchase_timestamp"]
13     },
14     "customers": {
15         "file_path": "../data/olist_customers_dataset.csv",
16         "primary_key": "customer_id",
17         "timestamp_column": "order_purchase_timestamp",
18         "columns": ["customer_id", "customer_unique_id", "customer_zip_code_prefix", "customer_city", "customer_state", "order_purchase_timestamp"]
19     },
20     "products": {
21         "file_path": "../data/olist_products_dataset.csv",
22         "primary_key": "product_id",
23         "timestamp_column": "order_purchase_timestamp",
24         "columns": ["product_id", "product_category_name", "product_name_lenght", "product_description_lenght", "product_photos_qty", "product_weight_g", "product_length_cm", "product_height_cm", "product_width_cm", "order_purchase_timestamp"]
25     },
26     "sellers": {
27         "file_path": "../data/olist_sellers_dataset.csv",
28         "primary_key": "seller_id",
29         "timestamp_column": "order_purchase_timestamp",
30         "columns": ["seller_id", "seller_zip_code_prefix", "seller_city", "seller_state", "order_purchase_timestamp"]
31     },
32     "reviews": {
33         "file_path": "../data/olist_order_reviews_dataset.csv",
34         "primary_key": "review_id",
35         "timestamp_column": "order_purchase_timestamp",
36         "columns": ["review_id", "order_id", "review_score", "review_comment_title", "review_comment_message", "review_creation_date", "review_answer_timestamp", "order_purchase_timestamp"]
37     },
38     "payments": {
39         "file_path": "../data/olist_order_payments_dataset.csv",
40         "primary_key": "order_id", # Note: payments doesn't have a unique payment_id, it's order_id + sequence
41         "timestamp_column": "order_purchase_timestamp",
42         "columns": ["order_id", "payment_sequential", "payment_type", "payment_installments", "payment_value", "order_purchase_timestamp"]
43     },
44     "geolocation": {
45         "file_path": "../data/olist_geolocation_dataset.csv",
46         "primary_key": "geolocation_zip_code_prefix",
47         "timestamp_column": "order_purchase_timestamp",
48         "columns": ["geolocation_zip_code_prefix", "geolocation_lat", "geolocation_lng", "geolocation_city", "geolocation_state", "order_purchase_timestamp"]
49     }
50 }
```